NLP-Based Interview Practice Chatbot with Automated Answer Evaluation

1. Title

InterviewMate: An NLP-Based Chatbot for Practicing Interview Answers with Automated Feedback

2. Problem Statement

Students preparing for job interviews often practice answering questions but have no reliable way to check whether their answers are relevant, complete, and well-structured before the actual interview. This project addresses this gap by building an NLP-based chatbot that evaluates a user's typed interview answer against a model answer and provides automated, structured feedback, allowing students to practice and improve independently.

3. Objectives

- To build a chatbot that asks interview questions from a curated question bank.
- To automatically evaluate the relevance of a user's answer using NLP techniques (TF-IDF and cosine similarity).
- To check coverage of key concepts expected in a good answer.
- To generate human-readable feedback highlighting missing concepts and filler-word usage.
- To evaluate how well the automated scoring aligns with human judgement.

4. Dataset

- **Dataset name:** Interview Q&A Dataset (self-curated)
- **Source:** Self-created by compiling common HR, technical, and behavioral interview questions from publicly available interview-preparation guides, combined with original model answers and key-concept lists written for this project.
- **Size:** 50 question–answer pairs.

- **Attributes:** `question_id`, `category` (HR / Technical / Behavioral / Domain), `question`, `model_answer`, `keywords` (expected key concepts, semicolon-separated).
- **Target variable:** Not a classification target in the traditional sense — the "target" is the degree of match (score 0–100) between a user's answer and the model answer/keywords.
- **Categories:** 4 (HR, Technical, Behavioral, Domain/Programming basics).
- **Limitations:**

Relatively small (50 pairs), English-only, model answers reflect one reasonable answer style rather than the only correct answer, so the system may unfairly penalize equally valid but differently-worded answers.

5. Methodology

User's typed answer → Preprocessing (lowercase, remove punctuation)

→ TF-IDF vectorization of [model answer, user answer]

→ Cosine similarity score

→ Keyword coverage check against expected key concepts

→ Filler-word detection

→ Weighted combined score (0-100) → Feedback generation

Stage explanation:

1. Preprocessing: The user's answer and the model answer are lowercased and stripped of punctuation to normalize the text before comparison.

Stopwords were intentionally not removed, since interview answers are short and stopwords can still contribute useful structural information to the TF-IDF vector.

2. TF-IDF + Cosine Similarity: For each question, a mini two-document corpus (model answer +user answer) is vectorized with TF-IDF, and cosine similarity measures how lexically similar the two answers are.

3. Keyword Coverage: Each question has a list of expected key concepts. The system checks how many appear in the user's answer, giving a coverage ratio.

4. Filler Word Detection: A simple regex-based count of common filler words (um, uh, like, basically, etc.) is used as a rough proxy for confidence/fluency.

5. Combined Score: `score = 0.6 × similarity + 0.4 × keyword_coverage`, scaled to 0–100.

Similarity was weighted slightly higher since candidates often use different keyword-free phrasing while still being conceptually correct.

6. Feedback Generation: The score is translated into a plain-language verdict, missing concepts are listed, and filler-word usage is flagged.

6. Implementation

- **Programming language:** Python
- **Libraries:** pandas, scikit-learn (`TfidfVectorizer`, `cosine_similarity`), `re`(regex)
- **NLP techniques used:** Text preprocessing, TF-IDF vectorization, cosine similarity, keyword/concept matching
- **Algorithm/model:** TF-IDF (a classical, explainable NLP technique — chosen over a deep learning model because the dataset is small and a simple, well-understood approach was preferred over a complex model that would be hard to justify with only 50 examples)
- **Key implementation details :**

- Per-question TF-IDF fitting (rather than one global vectorizer) keeps each comparison self-contained and avoids vocabulary mismatch issues across very different questions.

- The interactive chatbot loop (`run_chatbot()`) randomly samples questions each session.

- A separate batch evaluation function (`run_batch_evaluation()`) supports the quantitative evaluation described below.

7. Results

Live chatbot session — 5 questions answered:

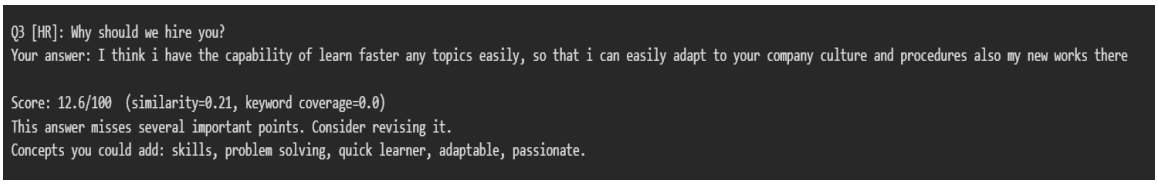
Q #	Category	Question	Score	Similarity	Keyword Coverage
1	Technical	Difference between array and linked list	68.9/100	0.481	1.0
2	Technical	Difference between list and tuple in Python	75.8/100	0.597	1.0
3	HR	Why should we hire you?	12.6/100	0.210	0.0
4	Technical	What is cosine similarity used for?	46.2/100	0.503	0.4
5	Technical	What is a hash table and how does it work?	60.6/100	0.477	0.8

Session average score: 52.8/100

High-scoring example (Q2 — list vs tuple): The answer used near-identical phrasing to the model answer ("mutable"/"immutable") and covered all expected keywords, producing a strong score of 75.8/100 with correct, encouraging feedback ("You covered all the expected key concepts.").

Low-scoring example (Q3 — "Why should we hire you?"): This is the most interesting result in the project. The answer ("I think i have the capability of learn faster any topics easily...") was a reasonable, genuine response, but it did not use any of the expected keywords (skills, problem solving, quick learner, adaptable,

passionate) and was phrased very differently from the model answer, so the similarity score was also low (0.21). The combined score of only 12.6/100 does not reflect the actual quality of the answer — it reflects the system's inability to recognize paraphrased meaning. This is discussed further in Limitations below.



Batch evaluation (system score vs. human judgement):

QID	Question	Score	System Bucket	Human Label	Match
26	Array vs linked list	68.9	Average	Good	No
16	List vs tuple	75.8	Good	Good	Yes
2	Why should we hire you?	12.6	Poor	Poor	Yes
31	Cosine similarity	46.2	Poor	Average	No
20	Hash table	60.6	Average	Average	Yes

```
QID  Score   Bucket    Human Label Match?
-------------------------------------------
26   68.9    average   good        No
16   75.8    good      good        Yes
2    12.6    poor      poor        Yes
31   46.2    poor      average     No
20   60.6    average   average     Yes
-------------------------------------------
Agreement with human labels: 60.0% (3/5)
```

Agreement with human labels: 60.0% (3/5)

Discussion of evaluation result: A 60% agreement rate shows the system is reasonably reliable at the extremes — it correctly identified the clearly strong answer (Q16) and the clearly weak answer (Q2) — but it disagreed with human judgement on borderline cases (Q26, Q31), where the answer was conceptually solid but the bucket thresholds (good ≥75, average 50–74, poor <50) were strict enough to downgrade a genuinely "good" human-rated answer to "average." This suggests the scoring thresholds, not just the similarity method, could be tuned further — a point carried into Future Scope below.

8. Limitations

- **TF-IDF cannot capture paraphrasing or synonyms.** A conceptually correct answer phrased differently from the model answer (e.g., "changeable" instead of "mutable") scores poorly, as seen in the Results section. This is the system's most significant limitation.
- The system cannot verify **factual correctness** — it only measures lexical/conceptual overlap with a reference answer, so a confidently wrong answer using the right keywords could still score well.
- Small, self-curated dataset (50 pairs) limits topic coverage.
- English-only; no support for code-mixed or regional-language answers.
- Filler-word detection is a crude heuristic, not a real measure of confidence or fluency.
- No handling of very short or off-topic answers beyond a low similarity/coverage score.

9. Future Scope

- Replace/augment TF-IDF with **Sentence-BERT embeddings** for semantic similarity that can recognize paraphrased but correct answers.
- Expand the dataset to 300+ questions across more domains (e.g., specific tech stacks).
- Add basic factual/keyword-weighted scoring so critical concepts count more than minor ones.
- Build a simple **Streamlit web interface** for a more polished, non-CLI user experience.
- Add speech-to-text input to simulate a real spoken mock interview.
- Track user progress over multiple sessions to show improvement trends over time.

10. Conclusion

This project developed InterviewMate, a chatbot-based NLP application that helps users practice interview questions and receive automated feedback on their answers. It applies core NLP techniques — text preprocessing, TF-IDF vectorization, and cosine similarity — combined with simple keyword-coverage checking to score answer quality. Testing showed that while the system is effective at catching answers that are clearly off-topic or incomplete, its reliance on TF-IDF means it struggles with correctly-phrased-but-differently-worded answers, a limitation directly tied to TF-IDF's lack of semantic understanding. This gap motivates the future use of transformer-based embeddings (e.g., Sentence-BERT) for a more robust evaluation.

Overall, the project demonstrates a complete, explainable NLP pipeline — from raw text input to a usable, interactive feedback tool — and highlights both the strengths and clear boundaries of classical NLP techniques for open-ended answer evaluation.